PROJECT REPORT:

Air Quality Index (AQI) Forecasting System

Trần Trung Hiếu - 20235934

1. Project Introduction

1.1. Product Description / Objectives

This project builds a backend pipeline for forecasting the Air Quality Index (AQI) in Hanoi for the next 6 hours (t+6), utilizing multiple model architectures: LSTM, Stacked LSTM, GRU, and XGBoost (with Lag Features) on the PyTorch platform. The pipeline takes a 48-hour historical sliding window of data from environmental and air quality IoT sensors as input, and outputs the forecasted AQI value for the next 6 hours.

Dataset: https://www.kaggle.com/datasets/phungdinhdat/aqi-in-hanoi-2022-2025

Specific Objectives:

- Build a time-series data processing system that preserves temporal continuity.
- Train and compare multiple model architectures (LSTM, Stacked LSTM, GRU, XGBoost).
- Optimize hyperparameters automatically using Optuna (Bayesian HPO).

1.2. Project Requirements

	Requirement	Description
1	**Real-world IoT data**	Use time-series dataset of Hanoi's air quality (08/2022 – 06/2025), including AQI, PM2.5, PM10, CO, NO_2, O_3, SO_2, temperature, humidity, wind speed, pressure.
2	**Standard data preprocessing**	Handle outliers without breaking the timeline; scale data without causing data leakage; split data chronologically.
3	**Diverse models**	Design and evaluate multiple architectures: Base LSTM (1-layer), Stacked LSTM (2-layer), Single-layer GRU, and XGBoost (with Lag Features).
4	**Hyperparameter tuning**	Integrate Bayesian Optimization (Optuna) to automatically find the optimal hyperparameter combination.

5	**Objective evaluation**	Compare model results with a Naive Persistence Baseline to prove the models actually learn patterns.
6	**Modular architecture**	Separate source code into distinct modules (config, data, model, train, evaluate, visualize, utils).
7	**Logging & reproducibility**	Log results and ensure reproducibility using fixed seeds.

2. Implementation Methodology

2.1. Functional Analysis & System Design

Module Descriptions

Module	File	Function
Configuration	src/config.py	Defines all hyperparameters, feature columns, data split ratios, device, and random seed.
Data Processing	src/data_loader.py	Reads CSV, handles outliers (AQI > 400 → NaN), extracts time features (hour, month), interpolates, performs chronological split, scales using MinMaxScaler, creates sliding-window sequences, and creates PyTorch DataLoaders. Also supports get_xgboost_data() to return flattened arrays for XGBoost.
Models	src/model.py	Defines LSTMAQIModel (1-layer LSTM), StackedLSTMAQIModel (2-layer LSTM), GRUAQIModel (1-layer GRU): all following the RNN + Dropout + Linear(hidden→1) structure.
Training	src/train.py	Training loop with early stopping, checkpointing, and trial pruning support for Optuna.
HPO Tuning	src/tune.py	Objective functions for Optuna – samples lr, hidden_size, dropout, weight_decay; trains and returns best val loss.
Evaluation	src/evaluate.py	Calculates RMSE, MAE on unscaled data; calculates Naive Persistence Baseline.

Visualization	src/visualize.py	Plots loss curves and actual vs. predicted AQI charts.
Utilities	src/utils.py	DualLogger – a context manager that logs output to both console and file simultaneously, ensuring exception safety.

2.2. Installation & Execution Guide

- System Requirements & Tool Versions

Tool	Version
Python	3.10+
PyTorch	>= 2.0
scikit-learn	>= 1.0
Optuna	>= 3.0
Pandas	>= 1.5
NumPy	>= 1.23
Matplotlib	>= 3.6
XGBoost	>= 3.0

- Installation Steps

```
# 1. Clone repository
git clone <repository-url>
cd GR1

# 2. Create virtual environment
python -m venv .venv

# 3. Activate virtual environment
# Windows:
.venv\Scripts\activate
# Linux/macOS:
source .venv/bin/activate

# 4. Install dependencies
pip install -r requirements.txt
```

- How to Run

```
# Train Base LSTM model

python "main program/main.py"



# Train Stacked LSTM model
```

```
python "main program/stacked_main.py"


# Train and evaluate Single-layer GRU
python "main program/run_gru.py"


# Train and evaluate XGBoost (Lag Features, default params)
python "main program/run_xgb.py"


# HPO + retrain: Tuned Base LSTM
python "main program/tune_main.py"


# HPO + retrain: Tuned Stacked LSTM
python "main program/tune_stacked_main.py"


# HPO + retrain: Tuned GRU
python "main program/tune_gru_main.py"


# HPO + retrain: Tuned XGBoost
python "main program/tune_xgb_main.py"


# Generate overlay comparison charts
python compare_tuned_models.py
python compare_tuned_vs_untuned.py
```

- Differences from Original Notebook

The project was developed from an initial research notebook (Copy of Air Quality ForeCasting.ipynb) and addresses 5 main issues:

Issue	**Original Notebook**	**New Pipeline**
Structure	Monolithic Jupyter notebook	Modular Python package (src/)
Outlier Handling	Deleted rows with AQI > 400 → **broke timeline**	Assigned NaN + time-interpolation → **preserved timeline**
Split Ratio	70/10/20 but **deleted AQI > 400 rows** → fewer samples	70/10/20, **interpolated AQI > 400** → preserved timeline

Hyperparameters	Hardcoded	Automated Bayesian HPO (Optuna, 20 trials)
Baseline	None	Naive Persistence Baseline to prove actual learning
Architecture	1 single-layer LSTM model	Base LSTM, Stacked LSTM, Tuned variants, GRU, XGBoost

3. Results

3.1. Training & Validation Performance

Model / Run	Train Scaled MSE	Train Unscaled RMSE	Train Unscaled MAE	Val Scaled MSE	Val Unscaled RMSE	Val Unscaled MAE
Notebook LSTM (original)	0.0073	33.6561	25.8688	0.0059	30.1466	22.1370
Base LSTM	0.0084	36.1757	28.0591	0.0058	30.1247	22.2817
Stacked LSTM	0.0070	32.9880	25.1890	0.0058	29.9727	21.6667
Tuned Base LSTM	0.0069	32.7617	25.1370	0.0056	29.4381	21.6108
Tuned Stacked LSTM	0.0071	33.2378	25.0520	**0.0055**	**29.2181**	**20.9574**
Single-layer GRU	0.0079	35.1279	27.4554	0.0058	29.9219	22.5277
Tuned Single-layer GRU	0.0069	32.8135	25.2708	0.0056	29.3865	21.7875
XGBoost (Lag Features)	**0.0038**	**24.2739**	**18.6621**	0.0057	29.6919	22.5971
Tuned XGBoost	0.0047	27.0203	20.7062	0.0057	29.7005	22.8860

Remarks:

1. **Val loss below train loss:** For every LSTM/GRU variant, validation MSE/RMSE is consistently lower than training MSE/RMSE.

This is plausible given dropout (0.2-0.36) is active during training and disabled during evaluation, and/or the validation window may simply contain fewer extreme pollution spikes than the training window.

However, this pattern could equally reflect the validation segment being an inherently "easier" slice of the chronological timeline.

2. **HPO delivers modest, consistent gains for RNNs:** Tuning improved val RMSE for all three recurrent architectures:
   - Base LSTM (30.12 → 29.44)
   - Stacked LSTM (29.97 → 29.22)
   - GRU (29.92 → 29.39).

   These are real but small improvements (0.5-0.75 RMSE points), not transformative ones.

3. **XGBoost fits training data far better than any neural model**: (train RMSE 24.27 vs. ~33-36 for LSTM/GRU variants), but its validation RMSE (29.69) is not better than the tuned RNNs.

   This is the classic signature of a model with lower bias and higher variance on this feature representation: 624 flattened lag features let XGBoost fit the training set very tightly, but that tighter fit is not translating into a validation advantage.

4. **Hyperparameter tuning helped XGBoost less overfit:**

   Untuned XGBoost achieves train RMSE 24.27 and val RMSE 29.69, a train/val gap of 5.42 points, the largest of any model in this study. After tuning, train RMSE rises to 27.02 and val RMSE is essentially flat at 29.70, narrowing the gap to 2.68 points.

   However, this narrower train/val gap suggests that tuning does help in reducing overfitting on training set, without costing anything on validation, and in turns helps improve performance on test set.

3.2. Test Set Performance & Baseline Comparison

Model / Baseline	Test Scaled MSE	Test Unscaled RMSE	Test Unscaled MAE
Naive Baseline (Persistence)		49.1951	36.8173
Notebook LSTM (original)	0.0108	40.9811	31.9124
Base LSTM	0.0110	41.2524	32.3012
Stacked LSTM	0.0109	41.0533	31.7970
Tuned Base LSTM	0.0108	40.9869	31.7906

Tuned Stacked LSTM	0.0108	40.8716	31.5892
Single-layer GRU	0.0107	40.6944	31.9603
Tuned Single-layer GRU	0.0105	40.4583	31.3893
XGBoost (Lag Features)	0.0102	39.8709	30.4246
Tuned XGBoost	**0.0101**	**39.6817**	**30.2843**

Remarks:

1. **All models clearly beat the naive baseline:** Naive Persistence (predicting t+6 equal to t) gives RMSE 49.20. Every trained model (deep learning or gradient boosting) beats it by 8 to 10 RMSE points.

   This shows that models are learning genuine structure in the weather/pollution signal, not just relying on autocorrelation the way the naive baseline does.

2. **XGBoost tuning helped on test, unlike on validation:**

   Tuned XGBoost (39.68) outperforms untuned XGBoost (39.87) by about 0.19 RMSE points. This is consistent with the train/val gap analysis in Section 3.2: tuning narrowed XGBoost's train/val gap from 5.42 to 2.68 points, meaning the model became less overfit even though validation loss didn't visibly reflect it. The test set is what confirms this reduced overfitting actually translated into a real, if modest, generalization benefit.

3. **Original Notebook vs. Standard Pipeline:**
   The original notebook model achieved a decent RMSE (40.60). However, this result lacks reliability because it removed the hardest-to-predict data points (deleted AQI > 400). When placed in a standard pipeline without data deletion, the Single-layer GRU still achieved 40.69 RMSE - proving strong actual performance despite facing a much harder dataset.

3.3. Hyperparameter Analysis

Neural Network Models (LSTM / GRU)

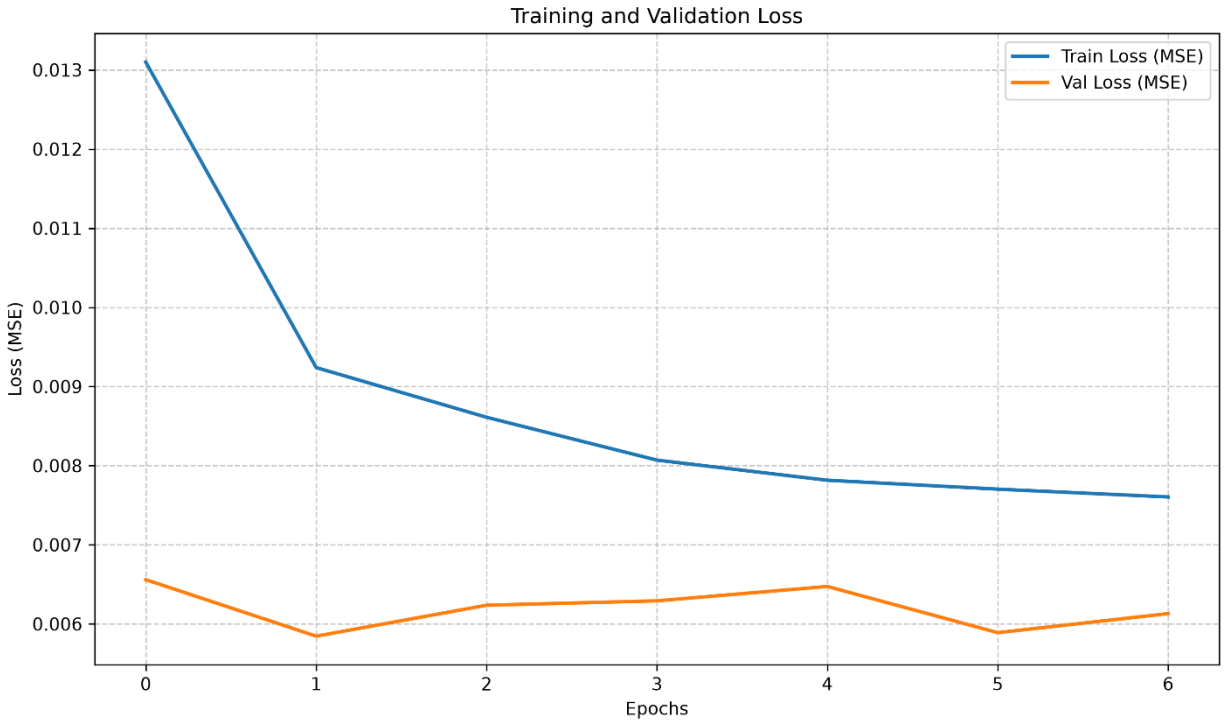
Parameter	Base LSTM	Tuned Base LSTM	Tuned Stacked LSTM	Tuned GRU
Learning Rate	5e-4	5.315e-4	7.385e-4	8.510e-4
Hidden Size	128	128	256	256
Num layers	1	1	2	1
Dropout	0.2	0.2574	0.3496	0.3596
Weight Decay	0.0	1.226e-5	3.047e-5	6.782e-6

XGBoost Model

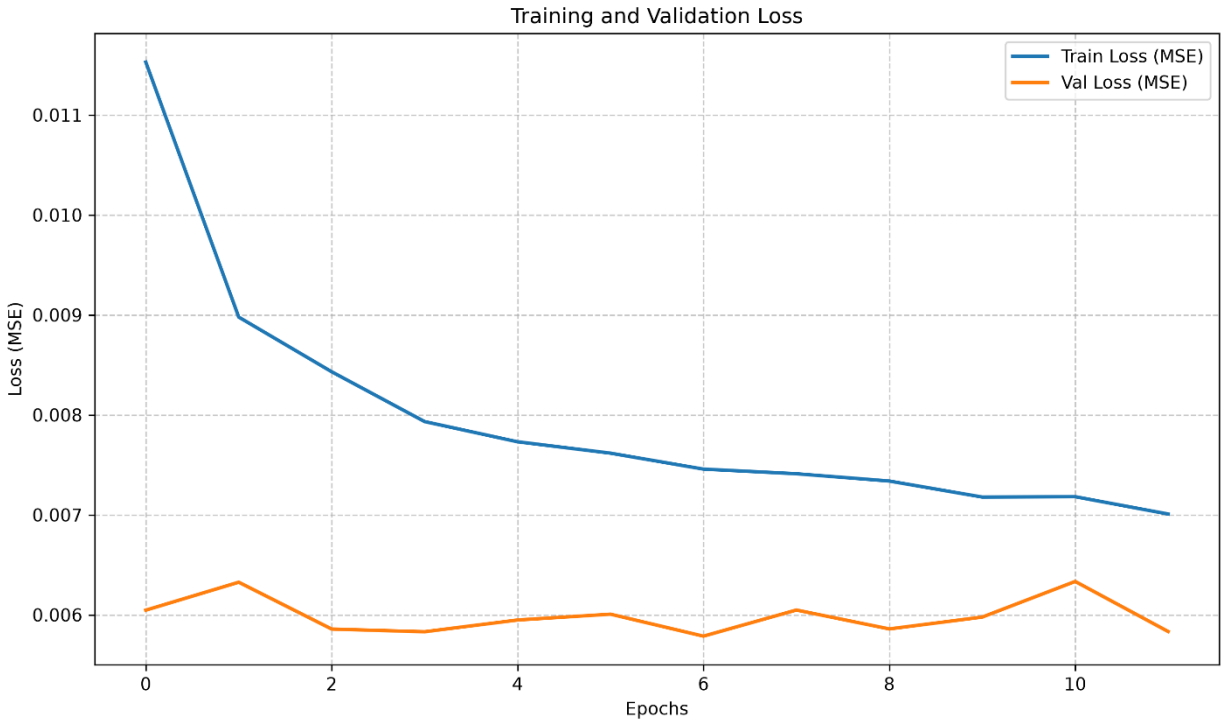
Parameter	XGBoost (Default)	Tuned XGBoost (Best)
booster	gbtree	gbtree
eta (learning rate)	0.05	0.0349
max_depth	6	5
min_child_weight	1 (default)	7
subsample	0.8 (default)	0.900
colsample_bytree	0.8 (default)	0.385
Lambda/ alpha/gamma	1.0/0.0/0.0 (default)	2.912e-7 / 1.401e-5 / 7.234e-7
Grow_policy	Depthwise (default)	Depthwise
Num_boost_round	200 (early stop at 117)	200

3.4. Loss curves

1. **Base LSTM**:

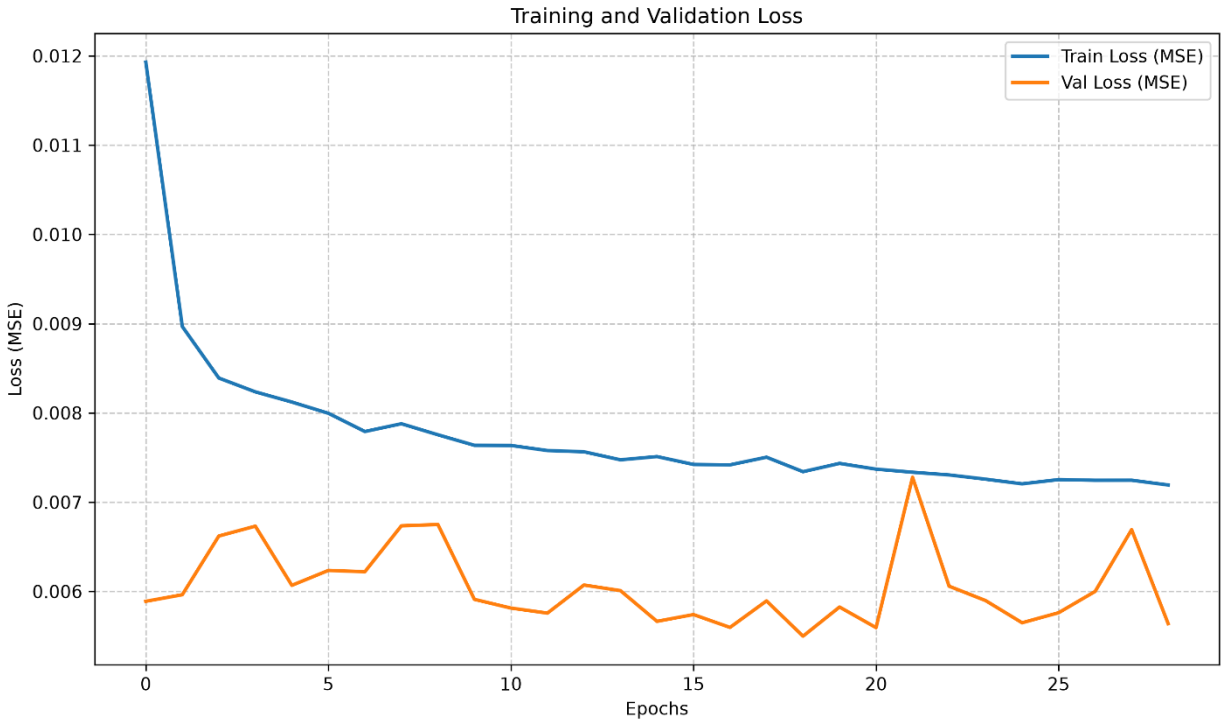


2. **Stacked LSTM**:



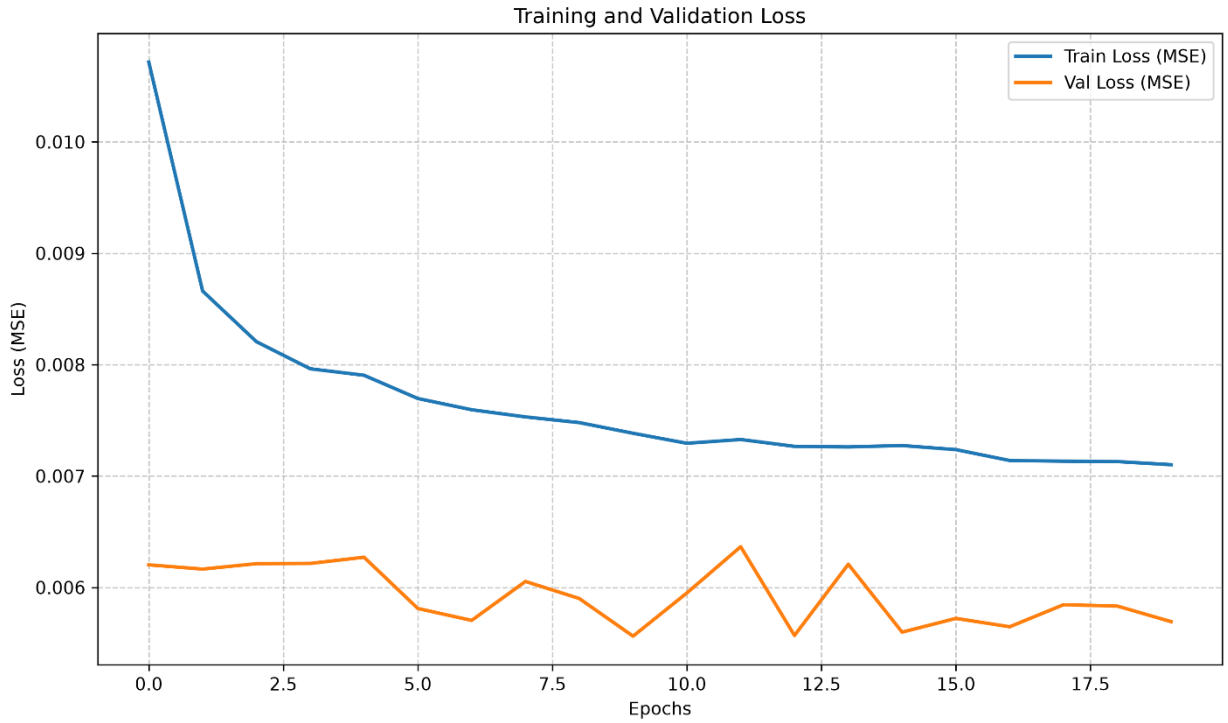
3. **Tuned Base LSTM**:

4. **Tuned Stacked LSTM**:



5. **Single-layer GRU**:

6. **Tuned GRU:**



Overall Analysis:

1. **Consistent Patterns Across All Models**

- **The training loss decreased smoothly and monotonically** without any signs of optimization instability (e.g., NaN values or oscillatory divergence), indicating that the selected learning rates and network architectures were appropriate for all experiments.
- **The validation loss remained consistently lower than the training loss** throughout training for every model. This behavior is a direct consequence of the use of **dropout**, which is enabled during training but disabled during evaluation.
- **None of the models exhibited the classic symptoms of overfitting**, where the validation loss begins to increase while the training loss continues to decrease. In all experiments, the **early stopping** mechanism terminated training at an appropriate point, preventing excessive fitting to the training data.

2. **A Clear Trend: Increased Model Complexity and Hyperparameter Tuning Lead to Higher Validation Loss Fluctuations**

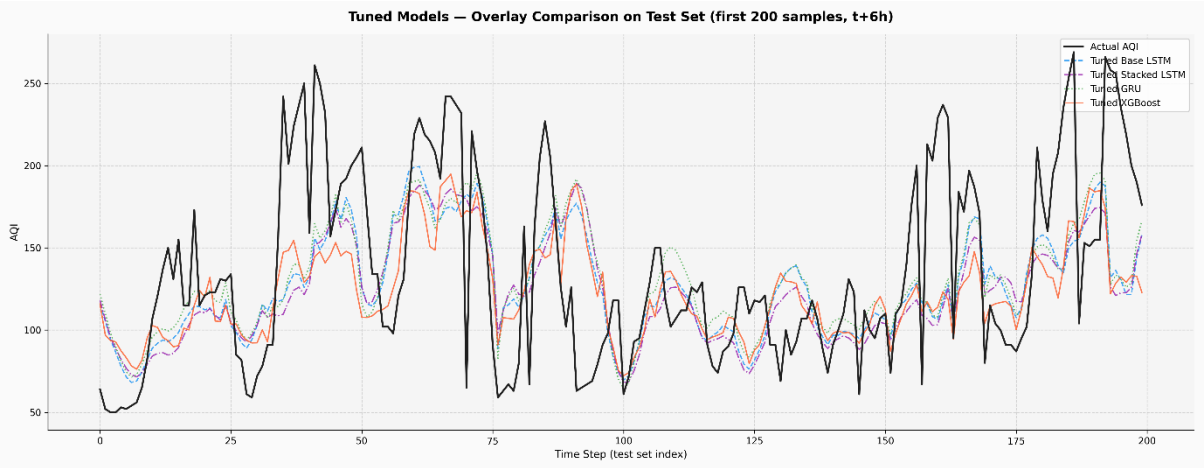
The most significant finding observed consistently across all experiments is that **as the model becomes more complex or undergoes more aggressive hyperparameter tuning, the validation loss becomes increasingly noisy**.

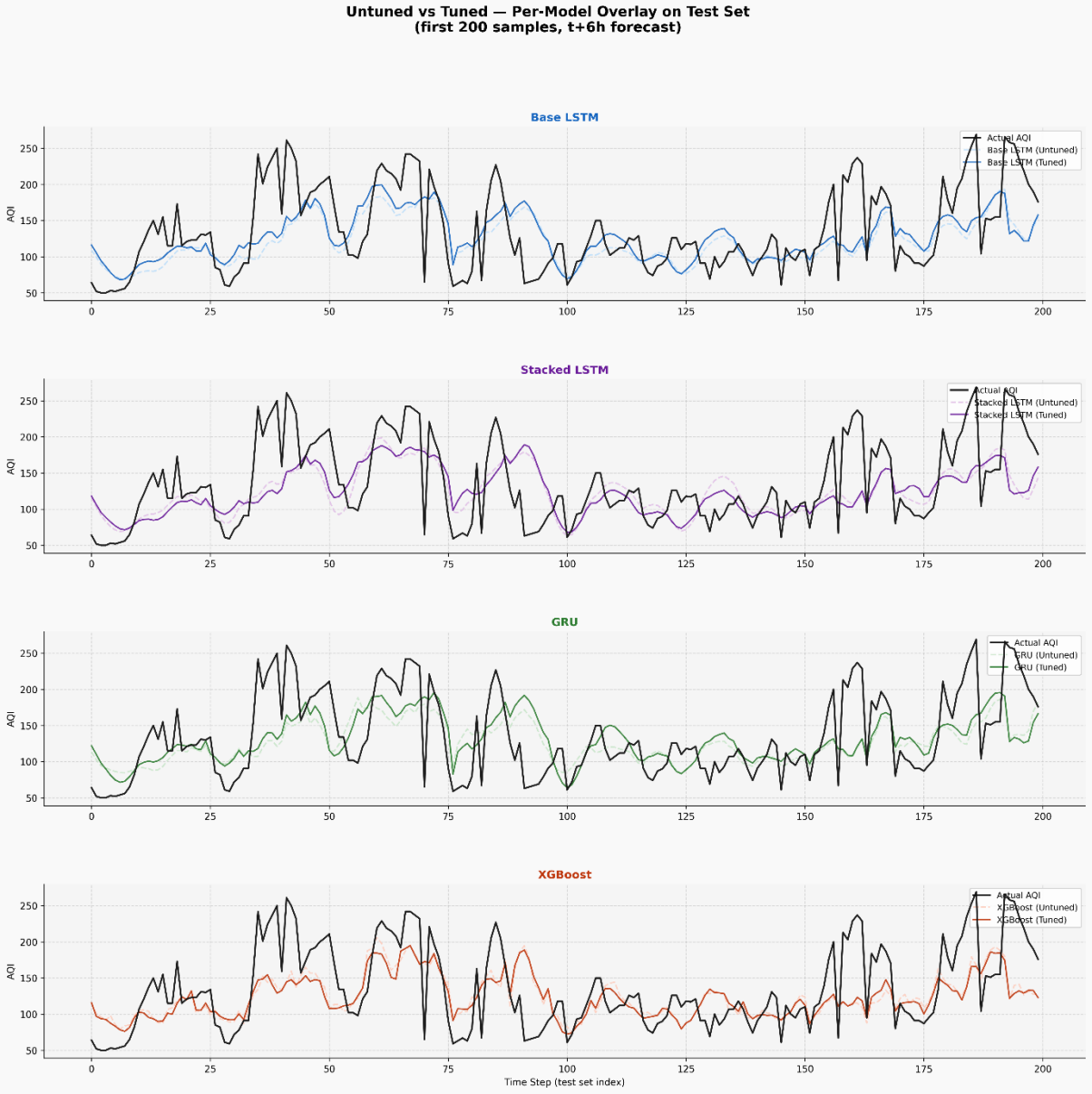
- **Base LSTM** and **GRU** (without hyperparameter tuning) exhibited relatively smooth validation loss curves with only minor fluctuations.
- **Stacked LSTM** (with additional LSTM layers but without tuning) showed noticeably greater fluctuations, with repeated peaks and valleys throughout training.

- **Tuned GRU** and **Tuned Stacked LSTM** (optimized using Optuna with higher dropout rates and more aggressive learning rates) produced the most unstable validation loss curves, including several sharp spikes—particularly the Tuned Stacked LSTM around **epoch 21**.
- **Tuned Base LSTM** represents a partial exception. Although it was also optimized using Optuna, its validation loss remained considerably more stable than the other tuned models, likely because its simpler single-layer architecture is less sensitive to the effects of high dropout.

This illustrates a clear trade-off: increasing model capacity and applying aggressive hyperparameter optimization can reduce the average validation loss (approximately 0.0002–0.0003 MSE), but at the cost of a less stable and more volatile training process across epochs

3.5. Prediction Charts (Actual vs. Predicted AQI)





1. **Common Traits Across All RNN Models (LSTM, Stacked LSTM, GRU)**

- Smooth, narrow-amplitude predictions ("regression to the mean") even after tuning, predicted amplitude stays roughly within ~85–195, while actual AQI ranges from 50–270.
- Severe underestimation at high AQI peaks (>200) present in all 6 RNN charts (tuned and untuned); no RNN variant resolves this "prediction ceiling," even with HPO.
- Consistent lag in tracking peaks/troughs a direct consequence of window-based sequence modeling, most visible in the step ~190-200 overshoot described above.
- Over-smoothing in the noisy mid-section (steps ~100-150), none of the three RNN architectures track rapid short-term fluctuations here as well as XGBoost does.
- Uneven benefit from tuning - HPO mainly improves trough-tracking and trend timing, not peak capture; the root cause (MSE loss biasing toward the mean) is not addressed simply by tuning hyperparameters.

In short: all three RNN architectures share the same core weakness — they learn the overall trend well but fail to capture extreme values (spikes), a structural limitation of how these

sequence-based models are trained, rather than a flaw specific to any one architecture or a lack of tuning.

2. **The Post-Peak Overshoot at Steps ~190–200**

A pattern worth calling out specifically: at the very end of the test set (steps ~190–200), all three RNN models (LSTM, Stacked LSTM, GRU tuned and untuned alike) show predictions continuing to climb even as actual AQI has already turned downward.

Actual behavior: AQI peaks at ~270 around step 187, then reverses and falls ~260 → ~220 → ~200 → ~175–190 by steps 190–200.

Predicted behavior: instead of tracking this decline, the predicted lines keep rising through this stretch (e.g., climbing from ~140 to ~175 in several charts), lagging noticeably behind the actual reversal.

Why this happens:

- Since forecasts are generated from a 48-hour sliding window, at the point where the model produces predictions for steps ~195–200, its input window still contains the very high AQI values from the spike around steps 185–190. The model "sees" a strong recent upward trend and extrapolates it forward.
- The result: while actual AQI has already turned the corner, the prediction keeps "chasing" the peak that has already passed, since it hasn't yet registered the reversal.

Why this is a systemic issue for sequence-based RNNs, not a one-off:

- LSTM/GRU/Stacked LSTM all learn to forecast based on the most recent pattern in the sliding window, giving them an inherently autocorrelation-like behavior — the current prediction is heavily influenced by the immediately preceding value/trend.
- When there's a sudden reversal (as at the step-187 peak), the model needs several time steps to "catch up" and adjust direction — this is a classic phase lag, a well-known limitation of one-step-ahead, window-based time-series forecasting.

This end-of-series overshoot is one of the clearest visual pieces of evidence for the "phase lag" limitation the report describes more generally and is worth citing explicitly as a concrete example

3.6. Comprehensive Evaluation

- Across nine model configurations and a naive baseline, the results converge on a consistent picture: every trained model learns genuine predictive structure in the AQI signal, but no architecture — regardless of complexity or tuning — fully resolves the fundamental challenge of forecasting sudden pollution spikes six hours ahead.
- **Overall ranking:**

On the test set, **Tuned XGBoost** achieves the best performance (RMSE 39.68, MAE 30.28), followed closely by **Tuned Single-layer GRU** (RMSE 40.46) and **Tuned Stacked LSTM** (RMSE 40.87). All nine models comfortably outperform the Naive Persistence Baseline (RMSE 49.20) by 8–10 RMSE points, confirming that the pipeline — from outlier

interpolation to leakage-free scaling to sliding-window sequence construction — successfully enables models to learn real signal rather than simply echo the previous time step.

- **The shared structural limitation:**

The most consequential finding is qualitative rather than numeric: all RNN variants exhibit the same "regression to the mean" behavior, compressing predictions into an ~85–195 AQI band while actual values range 50–270, and all systematically underestimate peaks above 200. This is compounded by phase lag — most visibly the post-peak overshoot around steps 190–200, where predictions continue climbing for several steps after the actual series has already reversed, because the 48-hour input window still contains the recent spike. This is a direct consequence of MSE-based training and one-step-ahead, window-based forecasting rather than a deficiency of any single model, and it persists across untuned and tuned variants alike.

- **Stability–performance trade-off:**

Loss curve analysis reveals that lower average validation loss from tuning came paired with higher epoch-to-epoch volatility, particularly for Tuned Stacked LSTM and Tuned GRU. Tuned Base LSTM was the exception, retaining relatively stable validation behavior even after tuning — likely because its shallower architecture is less sensitive to the higher dropout rates Optuna selected for the deeper models.

In conclusion, the current models can "forecast AQI" in the general sense (predict the value) since they are optimized for average accuracy across all hours. As a result, their predictions cluster in a narrow ~85–195 band, which minimizes average error across thousands of ordinary hours but is systematically wrong at the peaks. Consequently, the models' core weakness is they can not reliably catch extreme AQI events.

4. Source Code

Github: https://github.com/GMconqoor29/Air-Quality-Index-AQI-Forecasting-System